

Complexidade de Espaço

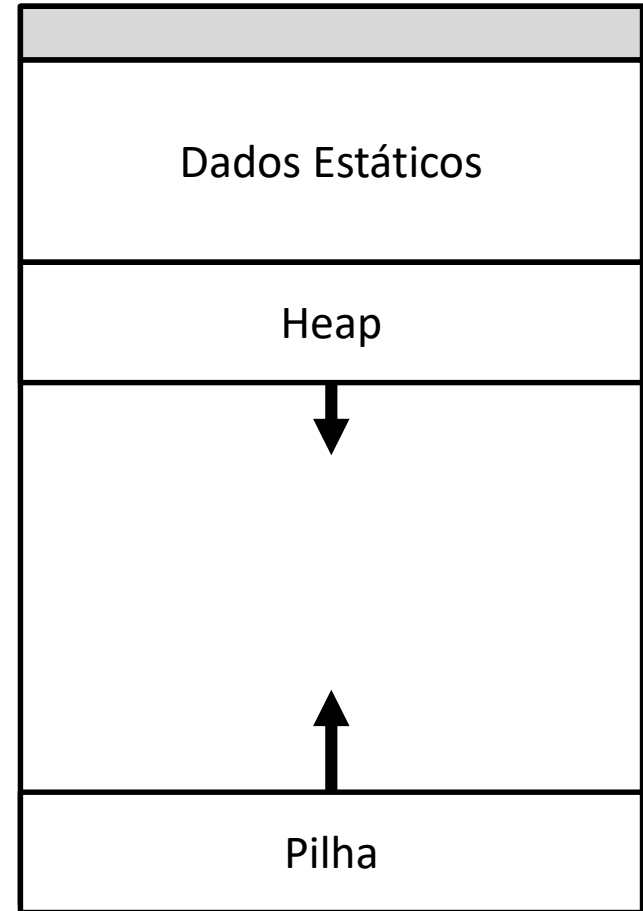
Prof. Dr. Leandro I. Pinto

Algoritmos Recursivos

- Algoritmos recursivos são mais simples de se fazer em muitos casos.
- Porém, há custos envolvidos.
- É necessário avaliar se um algoritmo recursivo é adequado em cada situação.

Memória do Programa

- Dados Estáticos: Variáveis estáticas, globais;
- Pilha: Armazena informações relacionadas a funções;
 - Endereço de retorno;
 - Parâmetros;
- Heap: Dados dinâmicos.
- Frame: uma região da pilha que armazena dados de uma função.



Fatorial Recursivo

```
1 public class Main
2 {
3     public static int fatRec(int n){
4         if(n <= 1){
5             return 1;
6         }
7         return n * fatRec(n-1);
8     }
9
10    public static void main(String[] args) {
11        int r;
12        r = Main.fat(5);
13        System.out.println(r);
14    }
15 }
```

javac Main.java
javap -c Main.class

```
public static int fatRec(int);
Code:
  0: iload_0
  1: iconst_1
  2: if_icmpgt      7
  5: iconst_1
  6: ireturn
  7: iload_0
  8: iload_0
  9: iconst_1
 10: isub
 11: invokestatic  #2
 14: imul
 15: ireturn
```

```
public static void main(jav
Code:
  0: iconst_5
  1: invokestatic  #3
  4: istore_1
  5: getstatic     #4
  8: iload_1
  9: invokevirtual #5
 12: return
```

Fatorial Recursivo

- Uma chamada recursiva é feita até n ser igual a 1.
- Portanto, temos que a complexidade de espaço é $O(n)$.

fat(3)

->fat(2)

->fat(1)

- Quanto a complexidade de tempo, vemos que uma chamada recursiva é feita para cada valor n .
Portanto é também $O(n)$.

Fatorial Não Recursivo

```
1 public class Main
2 {
3
4     public static int fat(int n){
5         int f = 1;
6         while(n > 1){
7             f = f * n;
8             n = n - 1;
9         }
10        return f;
11    }
12
13    public static void main(String[] args) {
14        int r;
15        r = Main.fat(5);
16        System.out.println(r);
17    }
18
19 }
```

```
public static int fat(int);
```

Code:

```
0: iconst_1
1: istore_1
2: iload_0
3: iconst_1
4: if_icmple      18
7: iload_1
8: iload_0
9: imul
10: istore_1
11: iload_0
12: iconst_1
13: isub
14: istore_0
15: goto          2
18: iload_1
19: ireturn
```

```
public static void main(jav
```

Code:

```
0: iconst_5
1: invokestatic  #3
4: istore_1
5: getstatic    #4
8: iload_1
9: invokevirtual #5
12: return
```

Fatorial Não Recursivo

- No fatorial não recursivo, a memória se mantém constante.
 - Portanto: A complexidade de espaço é $O(1)$
- Já o código se mantém com $O(n)$, assim como a versão recursiva.

Complexidade de Espaço

- Como a memória se comporta durante a execução do algoritmo?
 - Quantos frames durante uma entrada de tamanho n ?